

Московский физико-технический институт
Физтех-школа прикладной математики и информатики

АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ (ПРОДВИНУТЫЙ ПОТОК)

II СЕМЕСТР

Лектор: *Рухович Филипп Дмитриевич*



Автор: Влад Хохлов
Проект на GitHub

весна 2026

Содержание

1	Алгоритм Дейкстры	3
1.1	Обобщение на произвольную функцию длины пути	3
2	Теория игр	3
2.1	Ретроанализ	3
2.2	Сумма игр	3
3	DFS и его друзья	7
3.1	DFS	7
3.2	Реберная двусвязность	9
3.3	Вершинная двусвязность	11
3.4	Ориентированные графы	13
4	Дерево доминаторов	15
5	Динамическое программирование	17
5.1	НВП	17
5.2	НОП	17
5.3	Задача о рюкзаке	18
5.4	Остовные деревья	19

1 Алгоритм Дейкстры

1.1 Обобщение на произвольную функцию длины пути

2 Теория игр

2.1 Ретроанализ

Определение 2.1: Игра A называется:

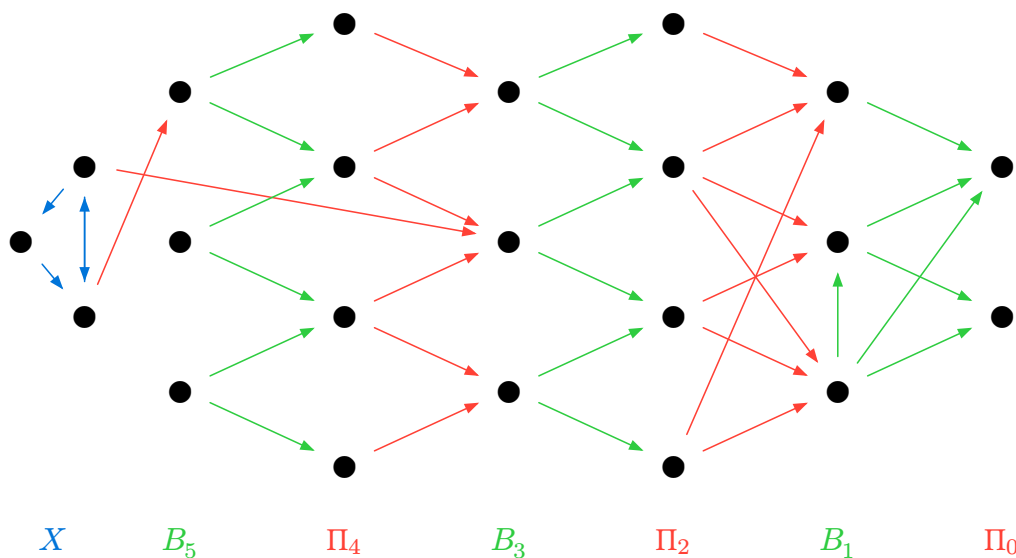
- **выигрышной**, если у первого игрока есть выигрышная стратегия
- **проигрышной**, если у второго игрока есть выигрышная стратегия
- **ничейной**, если у каждого игрока есть непроигрышная стратегия

Результат игры A : $\nu(A) \in \{\text{В}, \text{П}, \text{Н}\}$

Утверждение 2.1: Из $v \in X$:

1. всегда $\exists$ хотя бы один ход в $v' \in X$;
2. могут быть ходы в B_i ;
3. но нет ходов в Π_i .

Схема алгоритма ретроанализа:



2.2 Сумма игр

Определение 2.2: Суммой игр $A + B$ называется игра, в которой есть фишка в каждой игре, и игрок в свой ход двигает только одну фишку на свое усмотрение

Определение 2.3: Игры эквивалентны ($A \sim B$) если $\forall C$ — игра : $\nu(A + C) = \nu(B + C)$

Теорема 2.1: Если $A_1 \sim B_1$ и $A_2 \sim B_2$, то $A_1 + A_2 \sim B_1 + B_2$

Доказательство:

$$\forall C : \nu(A_1 + A_2 + C) = \nu(A_1 + (A_2 + C)) = \nu(B_1 + (A_2 + C)) = \nu(B_1 + (B_2 + C))$$

□

Теорема 2.2 (имени Л. Н. Толстого): Все проигрышные игры эквивалентны.

Доказательство: Разбор случаев: $\Pi + \Pi = \Pi$, $\Pi + \text{В} = \text{В}$, $\Pi + \text{Н} = \text{Н}$.

□

Замечание: Эта теорема носит имя Л. Н. Толстого, потому что его роман «Анна Каренина» начинается со слов: «Все счастливые семьи похожи друг на друга, каждая несчастливая семья несчастлива по-своему»

Определение 2.4: $*k$ – игра «Ним» с одной кучкой из k камней

Лемма 2.1: $*i + *j$ Π если $i = j$, иначе В

Теорема 2.3: $A \sim *i \Leftrightarrow A + *i$ Π

Доказательство:

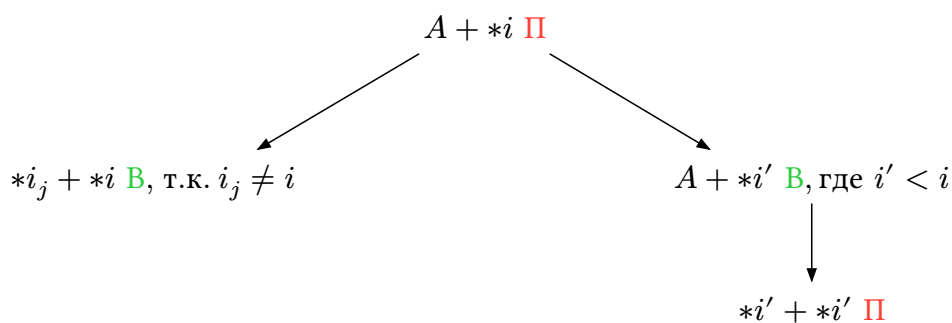
$$\Rightarrow: A \sim *i \stackrel{\text{T.1}}{\Rightarrow} A + *i \sim *i + *i \stackrel{\text{Л.1}}{=} \Pi$$

$$\Leftarrow: A + *i \stackrel{\text{T.2}}{\Rightarrow} A + *i \sim *0 \Rightarrow A \sim A + *0 \stackrel{\text{Л.1}}{\sim} A + (*i + *i) \stackrel{\text{T.1}}{\sim} *0 + *i \sim *i$$

□

Теорема 2.4 (Шпрага-Гранди): Пусть из игры A есть ходы в игры $A_1 \sim *i_1, A_2 \sim *i_2, \dots, A_k \sim *i_k$, тогда $A \sim *i$, где $i = \text{mex}\{i_1, i_2, \dots, i_k\}$

Доказательство:



□

Следствие: Если A ациклическая, то $A \sim *i$ для некоторого $i \geq 0$.

Определение 2.5: Если $A \sim *i$, то такое i – значение Шпрага-Гранди или Nimber для игры A . $\text{Nimber}(A) = i$.

Утверждение 2.2: $*i + *j + *k \text{ II} \Leftrightarrow i \oplus j \oplus k = 0$

Доказательство: По индукции по сумме $i + j + k$:

База:

- $i + j + k = 0 \Rightarrow i = j = k = 0 \Rightarrow i \oplus j \oplus k = 0$.
- В игре $*i + *j + *k = *0 + *0 + *0$ некуда ходить, значит она **II**.

Переход:

- Если $i \oplus j \oplus k = 0$, то любой ход ведет в игру $*i' + *j' + *k'$, где $i' + j' + k' < i + j + k$. Т.к. $i' \oplus j' \oplus k' \neq 0$, то по предположению индукции $*i' + *j' + *k'$ **B**.
- Иначе $i \oplus j \oplus k \neq 0$. Рассмотрим старший единичный бит в числе $i \oplus j \oplus k$. Тогда эта единица есть в каком-то из чисел i , j или в k , пусть без ограничения общности она есть в k . Сделаем $k' := i \oplus j$, тогда $k' < k$, потому что старший единичный бит в k заменился на 0. Тогда по предположению индукции, игра $*i + *j + *k'$ **II**, т.к. $i \oplus j \oplus k' = 0$, поэтому игра $*i + *j + *k$ **B**.

□

Теорема 2.5: $*i + *j \sim *(i \oplus j)$

Доказательство: $\stackrel{\text{Т.3}}{\Leftrightarrow} *i + *j + *(i \oplus j) \text{ II} \stackrel{\text{Утв.1}}{\Leftrightarrow} i \oplus j \oplus (i \oplus j) = 0$

□

Задача (Игра про Штирлица): Стоит очередь из $N \leq 5000$ людей. Рядом проходят Штирлиц и Мюллер и решают поиграть в игру. Они по очереди стреляют в одного человека. Кто не может выстрелить в человека, тот проигрывает. Если у какого-то человека в очереди убили всех соседей (у людей в центре 2 соседа, а у крайних людей 1 сосед), то такой человек убегает. Нужно определить кто выиграет при оптимальной игре.

Решение: Когда убивают человека в очереди, игра превращается в сумму двух независимых игр с меньшим количеством людей. Посчитаем Nimber каждой игры через mex Nimber-ов меньших игр.

$$\text{Nimber}[n] = \underset{i=0..n-1}{\text{mex}} \{ \text{Nimber}[i] \oplus \text{Nimber}[n-i-1] \}$$

□

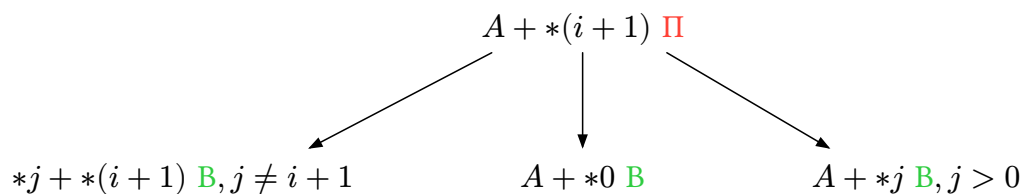
Задача (Игра «Малыш и Карлсон» или же «Молись и кайся»): Есть параллелепипед $A \times B \times C$. По очереди ходят 2 игрока - Малыш и Карлсон. Ход – разрезать параллелепипед на 2 части и съесть меньшую из них. Проигрывает тот, кто не может сделать ход. Определить победителя при оптимальной игре.

Решение: 3 независимые игры для каждой из сторон. Посчитаем Nimber отдельно для каждой стороны и возьмем их хог.

□

Задача (Игра «Hacking Bushes»): Есть подвешенное дерево. По очереди ходят 2 игрока. Ход – отрезать ребро и удалить подвешенное за это ребро поддерево, удалить корневую вершину никак не получится. Проигрывает тот, кто не может сделать ход. Определить победителя при оптимальной игре.

Решение: Научимся решать игру для «балалайки» A – дерева, в котором из корня идет только одно ребро. То есть A – вершина r , к которой присоединили произвольное дерево B , и r – корень в A . Пусть игра на дереве B эквивалентна $*i$. Докажем по индукции, что $A + *(i + 1)$ **П**.



Тогда $A \sim *(i + 1)$ из Т.3. Заметим, что обычное дерево – несколько балалаек, у которых объединили корни. То есть игра на обычном дереве – сумма независимых игр на его балалайках. Поэтому Nimber от обычного дерева равен тех его балалаек. Поэтому можно рекурсивно посчитать Nimber для всего дерева. $\square$

3 DFS и его друзья

3.1 DFS

Определение 3.1: Код алгоритма DFS:

```
enum EDFSColor {
    EDFS_WHITE // Белая вершина - Б
    EDFS_GRAY  // Серая вершина - С
    EDFS_BLACK // Черная вершина - Ч
};

struct Edge {
    int start, finish, id;
};

int currentTime_ = 0;

void dfs(int v, int parentEdgeId,
         const vector<vector<Edge>>& gr,
         vector<EDFSColor>& used,
         vector<int>& timeIn, vector<int>& timeOut) {
    used[v] = EDFS_GRAY;
    timeIn[v] = currentTime_++;

    for (Edge e : gr[v]) {
        int nv = e.finish;
        if (used[nv] == EDFS_WHITE) {
            dfs(nv, e.id, gr, used);
        }
    }
    used[v] = EDFS_BLACK;
    timeOut[v] = currentTime_++;
}

int main() {
    ...
    vector<EDFSColor> used(n, EDFS_WHITE);
    for (int v = 0; v < n; ++v) {
        if (used[v] == EDFS_WHITE) {
            dfs(v, -1, gr, used);
        }
    }
}
```

Определение 3.2: **Путь** – последовательность вершин и ребер, таких что каждая вершина является началом следующего ребра и концом предыдущего.

Определение 3.3:

- **Реберно-простой путь** – путь, в котором все ребра различны.
- **Вершинно-простой путь** – путь, в котором все вершины различны.

Определение 3.4: Вершина a **связана** с вершиной b , если существует путь от a до b .

Утверждение 3.1: В неориентированном графе связность – отношение эквивалентности на вершинах графа.

Определение 3.5: Компонента связности – это класс эквивалентности вершин графа относительно связности.

Лемма 3.1: Не существует момента времени в процессе DFS-а, когда какое-то ребро соединяет вершины Ч с Б.

Лемма 3.2 (О белых путях): Рассмотрим моменты времени в процессе DFS-а, когда вершина u была покрашена в серый цвет – момент t_1 , и после него в черный цвет – t_2 . Тогда:

1. Вершины $G \setminus \{u\}$, которые были С или Ч в t_1 , останутся таковыми и в t_2 ;
2. Вершины $G \setminus \{u\}$, бывшие Б в t_1 , в t_2 будут либо Ч, либо Б, причём такая v будет Ч $\Leftrightarrow$ от u до v в t_1 есть путь только по Б вершинам, не считая u .

Определение 3.6: Ребра дерева разделяются на:

- Ребра дерева DFS
- **Прямые**
- **Обратные**
- **Перекрестные**

Утверждение 3.2: Пусть ребро (u, v) – перекрестное, тогда $\text{timeIn}[u] > \text{timeIn}[v]$

Определение 3.7: $\text{timeSeg}[v] := [\text{timeIn}[v], \text{timeOut}[v]]$

Лемма 3.3: $\forall u, v \in V : \text{timeSeg}[u]$ и $\text{timeSeg}[v]$ либо не пересекаются, либо один вложен в другой. Более того, если (u, v) – ребро, то:

- a) (u, v) – **прямое** или дерева DFS $\Leftrightarrow \text{timeSeg}[v] \subset \text{timeSeg}[u]$
- b) (u, v) – **обратное** $\Leftrightarrow \text{timeSeg}[u] \subset \text{timeSeg}[v]$
- c) (u, v) – **перекрестное** $\Leftrightarrow \text{timeSeg}[u] \cap \text{timeSeg}[v] = \emptyset$

Утверждение 3.3: Так можно определить тип ребер:

```
void dfs(int v) {
    colors_[v] = EDFS_GRAY;
    timeIn_[v] = currentTime++;

    for (Edge e : graph_[v]) {
        int nv = e.finish;
        if (colors_[nv] == EDFS_WHITE) {
            // e - РЕБРО ДЕРЕВА DFS
            dfs(nv);
        }
        else if (colors_[nv] == EDFS_GRAY) {
            // e - ОБРАТНОЕ
        }
        else if (timeIn_[v] < timeIn_[nv]) {
            // e - ПРЯМОЕ
        }
        else {
            // e - ПЕРЕКРЕСТНОЕ
        }
    }

    timeOut_[v] = currentTime++;
    colors_[v] = EDFS_BLACK;
}
```

Утверждение 3.4: В графе есть цикл $\Leftrightarrow$ в результате DFS нашлось обратное ребро.

Доказательство:

- $\Leftarrow$: Очевидно
- $\Rightarrow$: Доказательство от противного: пусть обратного ребра нет. Тогда $\forall (u, v) \in E$ верно: $\text{timeOut}[u] > \text{timeOut}[v] \Rightarrow$ в графе есть топологическая сортировка. $\square$

Лемма 3.4: В неориентированном графе нет перекрестных ребер.

3.2 Реберная двусвязность

Замечание: Далее рассматриваем **неориентированный** граф $G = (V, E)$

Определение 3.8: Вершины u и v **реберно двусвязаны**, если $\exists$ два реберно-непересекающихся пути, соединяющих u и v .

Утверждение 3.5: Реберная двусвязность – отношение эквивалентности на вершинах.

Определение 3.9: Компонента реберной двусвязности – это класс эквивалентности вершин графа относительно реберной двусвязности.

Определение 3.10: Мост – ребро, удаление которого увеличивает количество компонент связности.

Утверждение 3.6: (u, v) – мост $\Leftrightarrow u$ и v не реберно двусвязаны.

Утверждение 3.7: Пусть u и v реберно двусвязаны, а $(u = a_1, a_2, \dots, a_k = v)$ – простой путь в дереве DFS, тогда вершины $a_1, a_2, \dots, a_k$ лежат в одной компоненте реберной двусвязности.

Доказательство: Рассмотрим ребро (a_i, a_{i+1}) . При его удалении можно построить путь от a_i до a_{i+1} следующим образом:

u и v реберно двусвязаны $\Rightarrow \exists$ путь от u до v , не проходящий через удаленное ребро (a_i, a_{i+1}) . От a_i до u и от a_{i+1} до v существует путь из дерева DFS. Соединяем эти 3 пути.

Тогда (a_i, a_{i+1}) – не мост $\Rightarrow a_i$ и a_{i+1} лежат в одной компоненте реберной двусвязности. Тогда по транзитивности все вершины $a_1, a_2, \dots, a_k$ лежат в одной компоненте реберной двусвязности. $\square$

Определение 3.11:

$$\text{upTime}[v] := \min_{u, w \in V} \{ \text{timeIn}[w] \mid (u, w) \text{ — обратное ребро, } u \text{ — потомок } v \text{ в дереве DFS} \}$$

Утверждение 3.8: Пусть (u, v) – ребро дерева DFS, v – ребенок u . Тогда

$$(u, v) \text{ — мост} \Leftrightarrow \text{upTime}[v] \geq \text{timeIn}[v]$$

Утверждение 3.9: Алгоритм поиска мостов и компонент реберной двусвязности:

```
stack<int> ST_;

void dfs(int v, int parentEdgeId) {
    colors_[v] = EDFS_GRAY;
    timeIn_[v] = currentTime_++;
    upTime_[v] = INF;
    ST_.push(v);

    for (Edge e : graph_[v]) {
        if (e.id == parentEdgeId) {
            continue;
        }
        int nv = e.finish;
        if (colors_[nv] == EDFS_WHITE) {
            dfs(nv, e.id);
            upTime_[v] = min(upTime_[v], upTime_[nv]);
        }
        else if (colors_[nv] == EDFS_GRAY) {
            upTime_[v] = min(upTime_[v], timeIn_[nv]);
        }
    }

    colors_[v] = EDFS_BLACK;
    timeOut_[v] = currentTime_++;

    if (upTime_[v] >= timeIn_[v]) {
        vector<int> comp;
        do {
            comp.push_back(ST_.top());
            ST_.pop();
        } while (comp.back() != v);
        // СОХРАНИТЬ comp КАК КОМПОНЕНТУ РЕБЕРНОЙ ДВУСВЯЗНОСТИ
    }
}
```

3.3 Вершинная двусвязность

Замечание: Далее рассматриваем неориентированный граф $G = (V, E)$ без петель.

Определение 3.12: Ребра (a, b) и (c, d) **вершинно двусвязаны**, если $\exists$ два вершинно-непересекающихся пути, соединяющих a с c и b с d или a с d и b с c .

Утверждение 3.10: Вершинная двусвязность – отношение эквивалентности на ребрах.

Определение 3.13: **Компонента вершинной двусвязности** – это класс эквивалентности ребер графа относительно вершинной двусвязности.

Определение 3.14: Вершина v – **точка сочленения**, если удаление v с инцидентными ей ребрами увеличивает количество компонент связности.

Утверждение 3.11: v – точка сочленения $\Leftrightarrow$ инцидентные ей ребра принадлежат хотя бы двум различным компонентам вершинной двусвязности.

Доказательство:

$\Rightarrow$: От противного: пусть инцидентные вершине v ребра принадлежат только одной компоненте вершинной двусвязности.

Т.к. v – точка сочленения, то $\exists a, b$: после удаления вершины v , вершины a и b перестанут быть связными. Значит $\nexists$ пути между a и b не через вершину v .

Ребра (v, a) и (v, b) – вершинно двусвязаны, поэтому $\exists$ путь между a и b не через вершину v , иначе бы все пути пересекались в вершине v – противоречие.

$\Leftarrow$: Рассмотрим 2 ребра (v, a) и (v, b) из разных компонент вершинной двусвязности, инцидентных вершине v . Тогда $\nexists$ пути между (a, b) не через вершину $v \Rightarrow$ при удалении вершины v с инцидентными ей ребрами вершины a и b перестанут быть связными $\Rightarrow$ в графе увеличится количество компонент связности $\Rightarrow v$ – точка сочленения. $\square$

Утверждение 3.12: Пусть e_1 и e_2 – ребра дерева DFS, принадлежащие одной компоненте вершинной двусвязности. Тогда все ребра пути в дереве DFS, соединяющем e_1 и e_2 , также лежат в этой компоненте.

Доказательство: Рассмотрим 2 случая:

1. e_1 и e_2 в одном поддереве
2. e_1 и e_2 в разных поддеревьях

$\square$

Следствие: Ребра дерева DFS, лежащие в одной компоненте вершинной двусвязности, образуют связное подвешенное дерево, у корня которого ровно 1 ребенок.

Утверждение 3.13: Ребро (u, v) дерева DFS, где u – родитель и v – ребенок, является корневым в своей компоненте вершинной двусвязности $\Leftrightarrow \text{upTime}[v] \geq \text{timeIn}[u]$.

Следствие: Алгоритм поиска компонент вершинной двусвязности:

```
vector<int> timeIn, timeOut, upTime;
vector<EDFSColor> colors;
stack<Edge> ST;
vector<vector<Edge>> graph;

void dfs(int v, int parentId) {
    colors[v] = EDFS_GRAY;
    timeIn[v] = currentTime++;
    upTime[v] = INF;

    for (Edge e : graph[v]) {
        if (e.id == parentId)
            continue;
        int nv = e.finish;
        if (colors[nv] == EDFS_WHITE) {
            ST.push(e);
            dfs(nv, e.id);
            upTime[v] = min(upTime[v], upTime[nv]);
            if (upTime[nv] >= timeIn[v]) {
                vector<Edge> comp;
                do {
                    comp.push_back(ST.top());
                    ST.pop();
                } while (comp.back().id != e.id);
                // сохранить comp как компоненту вершинной двусвязности
            }
        } else if (colors[nv] == EDFS_GRAY) {
            upTime[v] = min(upTime[v], timeIn[nv]);
            ST.push(e);
        }
    }
    colors[v] = EDFS_BLACK;
    timeOut[v] = currentTime++;
}
```

3.4 Ориентированные графы

Замечание: Далее рассматриваем **ориентированный** граф $G = (V, E)$

Утверждение 3.14: Пусть $\text{timeIn}[u] < \text{timeIn}[v]$. Тогда любой путь от u до v содержит какого-то общего предка u и v в дереве DFS.

Определение 3.15: Вершины u и v **сильно связаны**, если $\exists$ путь от u до v и от v до u .

Утверждение 3.15: Сильная связность – отношение эквивалентности на вершинах.

Определение 3.16: Компонента сильной связности (КСС) – класс эквивалентности относительно сильной связности.

Утверждение 3.16: Вершины одной КСС образуют в дереве DFS связное подвешенное дерево.

Определение 3.17: Введем новый цвет вершины – **фиолетовый (EDFS_PURPLE)**.

Определение 3.18: Новое определение upTime:

$$\text{upTime}[v] := \min_{u, w \in V} \{ \text{timeIn}[w] \mid (u, w) \text{ — обратное или перекрестное ребро,} \\ u \text{ — потомок } v \text{ в дереве DFS, } w \text{ — не фиолетовая при обработке } u \text{ DFS-ом} \}$$

Замечание: Код DFS для поиска КСС:

```
void dfs(int v) {
    colors[v] = EDFS_GRAY;
    timeIn, timeOut = ...
    upTime = ...
    ST.push(v);
    for (auto e : graph[v]) {
        int nv = e.finish;
        if (colors[nv] == EDFS_WHITE) {
            dfs(nv);
            upTime[v] = min(upTime[v], upTime[nv]);
        } else if (colors[nv] != EDFS_PURPLE) {
            upTime[v] = min(upTime[v], timeIn[nv]);
        }
    }
    colors[v] = EDFS_BLACK;

    if (v - КОРЕНЬ СВОЕЙ КСС) { // проверяем через следующее утверждение
        do {
            int u = ST.top(); ST.pop();
            comp.push_back(u);
            colors[u] = EDFS_PURPLE;
        } while (comp.back() != v);
    }
}
```

Утверждение 3.17: v – корень КСС $\Leftrightarrow \text{upTime}[v] \geq \text{timeIn}[v]$.

Утверждение 3.18: Алгоритм Косарайю для поиска КСС:

1. dfs1 находит топологическую сортировку по возрастанию timeOut: $v_1, v_2, \dots, v_n$.
2. Развернуть все ребра графа
3. Запускаем dfs2 от каждой вершины по убыванию timeOut: $v_n, v_{n-1}, \dots, v_1$

Тогда каждый запуск dfs2 обойдет все вершины ровно одной КСС и только их.

Замечание: Задачу 2-SAT можно решить за линейное время сведением к задаче о нахождении КСС.

4 Дерево доминаторов

Определение 4.1: **Граф потока управления (Flow graph):** $G = (V, E, r)$ – ориентированный граф, где r – корень, из r доступны все вершины $v \in V$.

Определение 4.2: Пусть $u, w \in V$. Тогда v **доминирует** над w ($v \text{ dom } w$), если $v \neq w$ и любой путь от r до w содержит v .

Утверждение 4.1: Если $u \text{ dom } v$ и $v \text{ dom } w$, то $u \text{ dom } w$

Утверждение 4.2: Если $u \neq v$, $u \text{ dom } w$ и $v \text{ dom } w$, то $u \text{ dom } v$ или $v \text{ dom } u$.

Следствие: $\forall w \in V \setminus \{r\} \exists! v :$

1. $v \text{ dom } w$
2. $\forall u \in V, u \text{ dom } w : u = v \vee u \text{ dom } v$

Определение 4.3: Такая вершина v из предыдущего следствия называется **непосредственным доминатором** $\text{idom}(v)$.

Определение 4.4: **Дерево доминаторов** графа потока управления $G = (V, E, r)$ – это (V, E_{idom}) , где $E_{\text{idom}} := \{(\text{idom}(w), w) \mid w \in V \setminus \{r\}\}$.

Замечание: Предполагаем, что в графе только **прямые ребра** и ребра дерева DFS.

Определение 4.5: **Полудоминатор** вершины w :

$$\text{sdom}(w) := \operatorname{argmin}\{\text{timeIn}[v] \mid (v, w) \in E - \text{прямое или дерева DFS}\}$$

Утверждение 4.3:

Пусть путь от $\text{sdom}(w)$ до w по ребрам дерева DFS выглядит как $\text{sdom}(w), v_1, v_2, \dots, v_k = w$.

Пусть $v_i := \operatorname{argmin}\{\text{timeIn}[\text{sdom}(v_j)] \mid j \in \{1, 2, \dots, k\}\}$, а $u_i := \text{sdom}(v_i)$.

Тогда:

1. Если $\text{timeIn}[u_i] \geq \text{timeIn}[\text{sdom}(w)]$, то $\text{idom}(w) = \text{sdom}(w)$
2. Иначе $\text{idom}(w) = \text{idom}(v_i)$

Замечание: Отменяем предположение про [прямые ребра](#), теперь в графе могут быть любые ребра.

Обозначение: Будем писать, что $v \xrightarrow{\bullet} w$, если v – предок w , и $v \xrightarrow{+} w$ если v – собственный предок w .

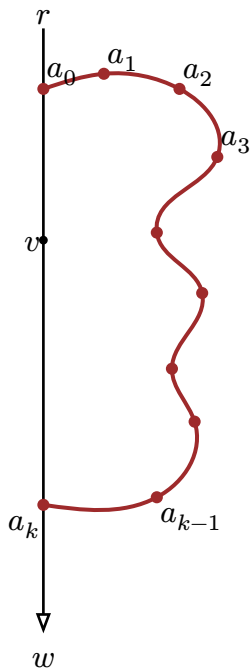
Утверждение 4.4: Пусть $v \xrightarrow{+} w$. Тогда

$v \text{ НЕ dom } w \iff \exists \text{ путь } (a_0, a_1, a_2, \dots, a_k), \text{ т.ч. :}$

1. $r \xrightarrow{\bullet} a_0 \xrightarrow{+} v \xrightarrow{+} a_k \xrightarrow{\bullet} w$;
2. $a_i \not\xrightarrow{\bullet} w, i = 1, 2, \dots, k-1$.

При этом в любом таком пути верно: $\text{timeIn}[a_i] > \text{timeIn}[a_k], i = 1, 2, \dots, k-1$

Доказательство: $v \text{ НЕ dom } w \iff$ существует [коричневый](#) обходной путь



□

Определение 4.6: Пусть $w \neq r$. Тогда **полудоминатор** вершины w :

$$\text{sdom}(w) := \operatorname{argmin}_{a_0 \in V} \{ \text{timeIn}[a_0] \mid \exists \text{ путь } (a_0, a_1, a_2, \dots, a_k = w), \text{ т.ч.} \\ \text{timeIn}[a_i] > \text{timeIn}[w], i = 1, 2, 3, \dots, k-1 \}$$

Утверждение 4.5: $\text{sdom}(w) \xrightarrow{+} w$

Доказательство:

1. $\text{timeIn}[\text{sdom}(w)] < \text{timeIn}[w]$, т.к. кандидатом на a_0 является родитель w .
2. Применяем Утв. 3.14 и получаем, что на минимальном пути из определения sdom ($\text{sdom}(w) = a_0, a_1, a_2, \dots, a_k = w$) лежит какой-то общий предок p вершин $\text{sdom}(w)$ и w .
3. Пусть $\text{sdom}(w) \not\xrightarrow{+} w$, тогда:
 - $p \neq \text{sdom}(w)$
 - $p \neq a_i, i = 1, 2, 3, \dots, k-1$, т.к. $\text{timeIn}[a_i] \stackrel{\text{sdom}}{>} \text{timeIn}[w] \geq \text{timeIn}[p] \stackrel{p \xrightarrow{+} w}{>} \text{timeIn}[p]$
 - $p \neq w$, иначе $\text{timeIn}[\text{sdom}(w)] > \text{timeIn}[w] = \text{timeIn}[p]$ – противоречие п.1

Получили противоречие п.2 $\Rightarrow \text{sdom}(w) \xrightarrow{+} w$. □

Утверждение 4.6: $\text{sdom}(w) = \operatorname{argmin} \{ \text{timeIn}[v] \mid v \in S_1(w) \cup S_2(w) \}$, где:

- $S_1(w) := \{v \mid (v, w) \text{ — прямое или дерева DFS} \}$
- $S_2(w) := \{ \text{sdom}(u) \mid (v, w) \text{ — перекрестное или обратное, } \text{timeIn}[u] > \text{timeIn}[w], u \xrightarrow{\bullet} v \}$

Утверждение 4.7: Поиск $\text{idom}(w)$ аналогичен поиску idom в Утв. 4.3.

5 Динамическое программирование

5.1 НВП

5.2 НОП

Утверждение 5.1: Поиск НОП строк a и b за $O(nm)$:

Динамика: $\text{dp}[i][j] := \text{НОП}(a[0\dots i-1], b[0\dots j-1])$

База: $\text{dp}[i][0] = \text{dp}[0][j] = 0$

Переход: $\text{dp}[i][j] = \begin{cases} 1 + \text{dp}[i-1][j-1] & \text{если } a[i] = b[j] \\ \max(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{иначе} \end{cases}$

В таком виде алгоритм использует $O(nm)$ памяти. Если хранить только последний слой, то памяти будет использоваться $O(m)$. Но чтобы в таком случае восстановить ответ, потребуется алгоритм Хиршберга.

Утверждение 5.2: $O(nm)$ – лучшая асимптотика по времени.

Доказательство: Пусть существует алгоритм, который выполнит сравнений на равенство символов меньше, чем nm . Тогда завалим его таким динамическим интерактором: на сравнение символов всегда отвечаем NO.

- Если алгоритм ответит 0, то интерактор скажет, что ответ был 1, т.к. 2 символа, которые алгоритм не сравнил были равны.
- Если алгоритм ответит 1, то интерактор скажет, что ответ был 0, т.к. любые 2 символа, которые алгоритм не сравнил были не равны. $\square$

5.3 Задача о рюкзаке

Задача: Дано N предметов: их веса и количества (w_i, k_i) . Надо ответить, возможно ли набрать веса $k \in [0, \dots, C]$ за $O(NC)$.

Решение:

$dp[i][w] := \min\{k \leq k_i \mid \text{можно набрать } w \text{ используя элементы первых } i \text{ типов,}$
 причем конкретно i -го типа берем ровно $k\}$

$dp[i][w]$ можно пересчитать через $dp[i-1][w]$ и $dp[i][w-w_i] + 1$ $\square$

Задача: $\sum_{i=1}^N w_i = C$. Надо ответить, возможно ли набрать веса $k \in [0, \dots, C]$ за $O\left(\frac{C\sqrt{C}}{32}\right)$.

Решение:

Количество различных w_i не превосходит $\sqrt{2C}$, поэтому объединим веса в пары (w_i, k_i) . Теперь каждую такую пару разобьем на новые веса $w_i, 2w_i, 4w_i, \dots, 2^{s_i}w_i, (k_i - 1 - 2 - \dots - 2^{s_i})w_i$.

Теперь оценим количество весов: рассмотрим все веса, которые мы записали с коэффициентом 2^k .

$$2^k w_{j_1} + 2^k w_{j_2} + \dots + 2^k w_{j_{t_k}} \leq C$$

$$w_{j_1} + w_{j_2} + \dots + w_{j_{t_k}} \leq \frac{C}{2^k}$$

Все эти веса w различные, поэтому $t_k \leq \sqrt{\frac{2C}{2^k}}$.

Количество всех новых весов: $\sum_{k=0}^{\infty} \sqrt{\frac{2C}{2^k}} = O(\sqrt{C})$. А такую задачу мы умеем решать за $O\left(\frac{C\sqrt{C}}{32}\right)$. $\square$

Задача: $\sum_{i=1}^N w_i \geq C, w_i \leq D$. Надо ответить, возможно ли набрать вес C за $O(ND)$.

Решение: Пусть $K \in [1 \dots N] : C - D \leq w_1 + \dots + w_K \leq C < w_1 + \dots + w_{K+1} \leq C + D$. Теперь делаем динамику $dp[l][r][w]$ – можно ли набрать $w \in [C - D, C + D]$, взяв веса

$w_1, \dots, w_{l-1}$, не взяв $w_{r+1}, \dots, w_N$, а $w_l, \dots, w_r$ можно брать как мы хотим. Эта динамика оптимизируется до $O(ND)$. $\square$

Определение 5.1: $(\max, +)$ -свертка (convolution):

Даны 2 массива $[a_0, a_1, \dots, a_{n-1}]$, $[b_0, b_1, \dots, b_{n-1}]$. Нужно посчитать массив $[c_0, c_1, \dots, c_{2n-2}]$, где $c[i] := \max_{j+k=i} (a_j + b_k)$.

Утверждение 5.3: Для полуограниченного рюкзака с $(w_i, cost_i, cnt_i)$:

$$dp[i] = (\max, +)\text{-conv}(dp[i-1], [0, -\infty, \dots, -\infty, cost_i, -\infty, \dots, -\infty, 2 - cost_i, \dots, cnt_i - cost_i])$$

Все $cost$ стоят на позициях $w_i, 2w_i, \dots, cnt_i w_i$.

Определение 5.2: Массив b выпукл вверх, когда $b_1 - b_0 \geq b_2 - b_1 \geq \dots \geq b_{n-1} - b_{n-2}$

Определение 5.3: Матрица $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ **монотонна**, если $(a \leq c) \rightarrow (b \leq d)$

Определение 5.4: Матрица $C_{N \times M}$ **монотонна**, если $\forall i_1, i_2, j_1, j_2, 1 \leq i_1 < i_2 \leq N, 1 \leq j_1 < j_2 \leq M$: матрица $\begin{pmatrix} C_{i_1, j_1} & C_{i_1, j_2} \\ C_{i_2, j_1} & C_{i_2, j_2} \end{pmatrix}$ монотонна.

5.4 Остовные деревья

Определение 5.5: **Разрез** (S, T) : $V = S \sqcup T$, где $S, T \neq \emptyset$, а V – множество вершин графа.

Определение 5.6: Ребро $e = (u, v)$ **пересекает** разрез, если $u \in S \oplus v \in S$

Лемма 5.1 (Лемма о безопасном ребре или теорема о разрезе): Пусть U – подмножество ребер какого-то мин. остова. Пусть (S, T) – разрез, который не пересекает ни одно ребро из U . Пусть e – минимальное по весу ребро, пересекающее (S, T) . Тогда множество $U \cup \{e\}$ является подмножеством ребер какого-то мин. остова.

Утверждение 5.4: Алгоритм Прима – почти ничем не отличается от алгоритма Дейкстры: добавляем в мин. остов минимальное ребро с его второй вершиной, торчащее из множества уже рассмотренных вершин.

Утверждение 5.5: Алгоритм Краскала – сортируем ребра по возрастанию и добавляем его, если не образовывается цикл. На цикл проверяем через СНМ.

Определение 5.7: $\log_2^* x := \min \left\{ k \mid \underbrace{\log_2 \log_2 \dots \log_2 x}_k \leq 1 \right\}$